GUIA DO ALUNO

Câmera Tapo C200/TC70: instalar, monitorar, gravar — e fazer a casa reagir

Instalação e posicionamento · app Tapo · rede local, nuvem e microSD · RTSP/ONVIF no PC · ESP32 + Python: gestos e reconhecimento facial acendem luzes e alarmes

---

Câmera: TP-Link Tapo C200 (modelo TC70 no Brasil) — Full HD, pan 360°/tilt 114°, visão noturna, áudio bidirecional

Projeto final: câmera + ESP32 na mesma rede: gesto liga a luz; rosto conhecido = boas-vindas; desconhecido = alarme

Pré-requisitos: aulas de LEDs/ESP32 web e de ESP32-CAM (OpenCV/MediaPipe) • **Carga horária:** 6 aulas de 100 min

Espaço Cria — 2026

Sumário

Como este curso funciona

Segurança, privacidade e ética — antes de ligar

Aula 1 — Conhecendo e instalando a Tapo C200/TC70

Aula 2 — Monitorando pelo celular

Aula 3 — Gravar e armazenar: microSD, nuvem e PC

Aula 4 — A câmera na rede: RTSP, ONVIF e segurança

Aula 5 — ESP32 atuador: luz, alarme e LED comandados pela rede

Aula 6 — Projeto: gestos e reconhecimento facial acionam o ESP32

Para ir além

Glossário

Anexo — Checklist de instalação da câmera

Como este curso funciona

Nas aulas anteriores construímos câmeras com o ESP32-CAM. Agora vamos instalar uma **câmera comercial** — a Tapo C200/TC70, a mais vendida no Brasil — e descobrir que ela faz muito bem o que custa caro fazer na mão (vídeo Full HD estável, pan/tilt motorizado, visão noturna, app, nuvem) e que, mesmo sendo um produto fechado, ela **fala protocolos abertos** (RTSP e ONVIF). É por essa porta que vamos ligá-la ao nosso ESP32: o PC lê o vídeo, reconhece gestos e rostos, e manda o ESP32 acender a luz ou tocar o alarme. O objetivo de monitoramento é concreto: **as laterais direita e os fundos da sala**.

Aula	Tema	Você vai fazer	Arquivos
1	Conhecendo e instalando	Entender o hardware e a física; escolher o ponto de instalação; parear no app	—
2	Monitorar pelo celular	Ao vivo, pan/tilt, presets de patrulha, detecção de movimento, notificações, áudio, modo privacidade	—
3	Gravar e armazenar	microSD (local), Tapo Care (nuvem), gravação no PC por RTSP (NVR caseiro); exportar; LGPD	tapo_ver.py, tapo_gravar.py
4	A câmera na rede: RTSP/ONVIF	Conta da câmera, VLC, OpenCV; rede local × internet; segurança	tapo_ver.py
5	ESP32 atuador	Relé (luz), buzzer, LED RGB e botão comandados por HTTP na mesma rede	01_esp32_luzes_alerta
6	Projeto: gestos e rostos	Python vê a Tapo → MediaPipe + reconhecimento facial → ESP32 age	cadastrar_rosto.py, tapo_gestos_faces.py

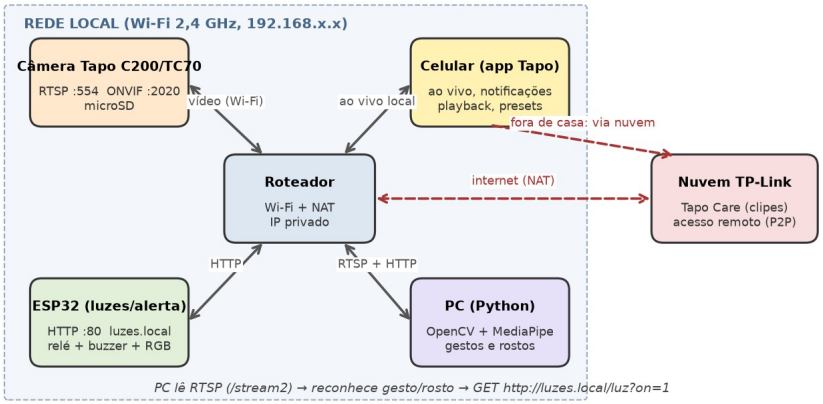


Figura 1 — Quem fala com quem: tudo na rede local, exceto a nuvem TP-Link (acesso remoto e Tapo Care).

Como usar este guia

As Aulas 1–4 são **tutorial** (passo a passo no app e no PC, com perguntas para pensar); as Aulas 5–6 são **projeto** (código + montagem + desafios com lógica e alertas). Os caminhos de menu do app Tapo podem mudar de nome entre versões — o guia usa os nomes da versão atual; procure pelo ícone/termo equivalente.

Códigos em `aula-tapo/codigo/` (ESP32) e `aula-tapo/python/` (PC). Modelos de IA em `python/modelos/`.

Segurança, privacidade e ética — antes de ligar

- A câmera **grava pessoas**. Instale só onde a turma/escola combinou (laterais e fundos da sala, nunca banheiros, vestiários ou janelas de vizinhos). Avise com um cartaz “ambiente monitorado”.
- Conta TP-Link **do laboratório**, senha forte e **verificação em duas etapas**. A senha da “conta da câmera” (RTSP) é outra — não use a mesma.
- Rede Wi-Fi com senha WPA2/WPA3; se possível uma rede separada para dispositivos IoT. **Nunca** abra a porta 554 da câmera para a internet.
- Imagens gravadas (microSD, nuvem, PC) são dados pessoais (LGPD): defina quem acessa, por quanto tempo ficam e quando são apagadas.
- Relé e rede elétrica: na aula, a “lâmpada” é um LED/lâmpada de 12 V. Ligar 127/220 V no relé **só com o professor**, caixa fechada e disjuntor desligado.

Aula 1 — Conhecendo e instalando a Tapo C200/TC70

O que você vai aprender

- O que há dentro de uma câmera IP: sensor, lente, motores, LEDs infravermelhos, microfone/alto-falante, Wi-Fi, microSD.
- A física da imagem noturna (infravermelho) e do campo de visão.
- Como escolher o lugar: altura, ângulo, luz, Wi-Fi, tomada — para cobrir **lateral direita e fundos**.
- Parear a câmera com o app Tapo e fazer a configuração inicial.

Ficha técnica (verifique a etiqueta: a versão de hardware muda alguns números)

Item	Tapo C200 / TC70	O que significa na prática
Resolução	1080p Full HD (1920×1080), 15–30 fps conforme a versão	Dá para ler um crachá a ~3 m; não dá para ler placa a 15 m
Lente / campo	4 mm, ~65° na horizontal	Com o pan/tilt o campo total vira 360° × 114°
Pan / tilt	360° horizontal, 114° vertical, motorizado	Presets e patrulha automática cobrem a sala inteira
Visão noturna	LEDs IR 850 nm, 9–12 m	Imagem em preto e branco no escuro; reflete em vidro
Áudio	Microfone + alto-falante (bidirecional)	Falar com quem está na sala pelo celular
Detecção	Movimento com zonas e sensibilidade; som (versões novas)	Notificação no celular
Armazenamento	microSD até 128 GB (v1) / 256–512 GB (versões novas); nuvem Tapo Care	Local × nuvem — Aula 3
Rede	Wi-Fi **2,4 GHz** apenas (802.11 b/g/n)	Não aparece em redes só 5 GHz
Protocolos abertos	RTSP (porta 554), ONVIF Profile S (porta 2020)	É por aqui que o PC/ESP32 entram — Aula 4
Alimentação	9 V 0,6 A (fonte própria) — versões novas 5 V USB-C	Precisa de tomada perto
Privacidade	Modo privacidade (lente vira para dentro) e LED de status	Dá para “fechar” a câmera à distância

A física por trás

- **Sensor CMOS + lente**: como no ESP32-CAM, mas com processador de imagem dedicado (WDR, redução de ruído) e codificação **H.264** — 1080p cabe em 1–2 Mbit/s de Wi-Fi (o MJPEG do ESP32-CAM gastaria 10×).
- **Visão noturna**: um sensor de luz desliga o filtro IR e acende LEDs de **850 nm** (invisíveis ao olho, mas o sensor enxerga). Por isso a imagem fica em preto e branco e objetos quentes **não** aparecem “brilhando” — não é térmica, é iluminação IR refletida. Vidro reflete os LEDs: nunca aponte a câmera através de uma janela à noite.

- **Campo de visão**: ~65° com lente de 4 mm; a 5 m de distância cobre ~6 m de largura. Instalada no canto, cobre as duas paredes adjacentes.
- **Pan/tilt**: dois motores de passo; a câmera lembra posições (**presets**) e pode passear por elas (**patrulha**).

Escolhendo o ponto de instalação (laterais direita e fundos)

1. Desenhe a planta da sala com a porta, as janelas, as mesas e a tomada mais próxima.
2. **Canto do fundo esquerdo, no alto (2,2–2,6 m), inclinada ~20° para baixo**: desse canto, girando o pan, a câmera vê o fundo inteiro e toda a lateral direita; a porta fica no campo diagonal. Alternativa: centro da parede dos fundos, se a lateral direita for curta.
3. Evite: contraluz (janela atrás do que se quer ver), ventilador/ar-condicionado no quadro (movimento falso), lâmpada apontada para a lente, lugares de passagem onde esbarrem.
4. Wi-Fi: no celular, meça o sinal no ponto escolhido (≥ 2 barras de 2,4 GHz). Paredes de concreto derrubam o sinal.
5. Suporte: parafusos/buchas na parede ou prateleira; a base tem furos e a câmera pode ficar **de cabeça para baixo** (o app inverte a imagem).

Pareando no app Tapo

1. Instale o app **Tapo** (Android/iOS) e crie uma **ID TP-Link** (e-mail do laboratório). Ative a verificação em duas etapas.
2. Ligue a câmera na fonte; espere o LED piscar **vermelho e verde** (modo de pareamento, ~30 s). Se não piscar, segure RESET por 5 s.
3. No app: **+ → Câmeras → Tapo C200**. O celular conecta temporariamente ao Wi-Fi da câmera (`Tapo_Cam_XXXX`) e depois você informa a rede **2,4 GHz** da sala e a senha.
4. Dê um nome (“Sala – fundos”), escolha o local, atualize o **firmware** quando o app oferecer (corrige falhas de segurança).
5. Configurações iniciais: fuso horário, **inverter imagem** se instalada de cabeça para baixo, LED de status (desligue para não chamar atenção à noite), modo noturno automático.

Experimente

1. Com a sala escura, observe o LED IR vermelho fraco na câmera e a imagem P&B. Aponte para a janela: vê o reflexo?
2. Meça a largura real do campo a 3 m com uma trena e compare com $2 \times 3 \times \tan(65°/2) \approx 3{,}8$ m.
3. Gire o pan até o limite: 360° é uma volta completa? (Há um batente; a câmera volta pelo outro lado.)

Desafios

1. **1.1** Desenhe na planta o “cone” de visão da câmera em 3 presets (fundos, lateral direita, porta). Alguma área fica cega? Onde colocaria uma segunda câmera?
2. **1.2** Calcule quantos pixels um rosto de 15 cm ocupa a 3 m e a 8 m (campo de ~3,8 m a 3 m; 1920 px). A partir de quantos pixels dá para reconhecer alguém? (~60 px entre os olhos, diz a literatura.)
3. **1.3** Teste a visão noturna com um objeto preto fosco e um branco: por que o preto “some”?

Aula 2 — Monitorando pelo celular

O que você vai aprender

- Ao vivo na rede local × pela internet: como o app encontra a câmera (P2P pela nuvem TP-Link).
- Pan/tilt, presets, patrulha, zoom digital, áudio bidirecional.
- Detecção de movimento (zonas, sensibilidade, horário), notificações e modo privacidade.
- O que a câmera **não** faz sozinha (e por que vamos precisar do PC + ESP32).

Ao vivo: rede local × internet

Quando o celular está no **mesmo Wi-Fi**, o app fala direto com a câmera (IP privado 192.168.x.x): latência de ~0,5 s. Fora de casa, o app não consegue chegar ao IP privado (NAT, como vimos no ESP32-CAM) — então a câmera mantém uma conexão de saída com a **nuvem TP-Link**, que apresenta o celular à câmera (**P2P**) ou retransmite o vídeo. Nada de abrir portas no roteador. Teste: desligue o Wi-Fi do celular (use 4G) e veja o ao vivo funcionar com um pouco mais de atraso.

Passo a passo no app

1. **Ao vivo**: toque na câmera; arraste na imagem (ou use as setas) para o pan/tilt; pinça = zoom digital; ícone do microfone = falar; alto-falante = ouvir.
2. **Presets**: na tela ao vivo → *Pan & Tilt* → *Predefinições* → aponte para os **fundos** e salve "Fundos"; aponte para a **lateral direita** e salve "Lateral D"; mais um para a **porta**.
3. **Patrulha**: *Patrulha* → ative e escolha os presets e o intervalo (ex.: 30 s). A câmera passa a "vigiar" a sala inteira.
4. **Detecção de movimento**: *Configurações → Detecção* → ative; *Zonas de atividade*: desenhe só a lateral direita e os fundos (ignore o corredor). *Sensibilidade*: média. *Programação*: só fora do horário de aula, por exemplo.
5. **Notificações**: ative no app e no celular; *Notificações ricas* enviam uma miniatura.
6. **Modo privacidade**: botão na tela ao vivo — a lente gira para dentro da carcaça e o LED apaga. Combine a regra: durante a aula, privacidade **ligada**.
7. **Modo noturno**: automático; teste "desligado" para ver o quanto a imagem piora.
8. **Compartilhar**: *Configurações → Compartilhamento do dispositivo* → convide outra ID TP-Link (o professor) com permissão só de visualização.

O que a câmera faz e o que não faz

Faz: ao vivo, gravação por evento, pan/tilt programado, notificação de movimento (e de pessoa/som em versões novas), áudio, privacidade.

Não faz: reconhecer gestos, reconhecer **quem** é a pessoa (só "pessoa"), acionar luzes/relés/buzzers fora do ecossistema Tapo, rodar o seu código. Para isso precisamos de um **cérebro externo** (PC com Python) e de **mãos** (ESP32) — Aulas 5 e 6.

Experimente

1. Deixe a patrulha ligada 10 min e conte as notificações falsas (ventilador, cortina, luz). Ajuste zonas e sensibilidade até zerar.

2. Fale pelo app com um colega na sala: qual o atraso? Pela internet (4G) o atraso aumenta?
3. Compare o ao vivo em Wi-Fi e em 4G: a qualidade cai? O app reduz a resolução automaticamente?

Desafios (lógica)

1. **2.1** Projete a **rotina** ideal da câmera da sala: tabela horário × modo (privacidade, patrulha, detecção, notificação) para dia de aula, noite e fim de semana. Configure no app o que for possível.
2. **2.2** Zonas de detecção: por que detectar “só a lateral direita” reduz alarmes falsos? Desenhe a zona e justifique.
3. **2.3** Teste cego: um colega entra pela porta sem avisar; quem recebe a notificação cronometra o tempo até o celular tocar. O que domina esse tempo?

Aula 3 — Gravar e armazenar: microSD, nuvem e PC

O que você vai aprender

- As três formas de guardar vídeo e os prós/contras de cada uma.
- Gravação contínua × por evento; gravação em loop; quanto cabe num cartão.
- Exportar clipes para o celular; o que a nuvem Tapo Care oferece.
- Montar um **NVR caseiro** no PC gravando por RTSP (Python).

	microSD (na câmera)	**Nuvem Tapo Care**	**PC/NAS por RTSP (Aula 4)**
Custo	cartão (R$ 40–120)	assinatura mensal/anual (teste grátis de 30 dias)	PC ligado + disco
O que grava	contínuo **ou** eventos, 24 h	clipes de **eventos** (7 ou 30 dias conforme o plano; planos de 180 dias em alguns países)	o que você programar (contínuo, movimento, horário)
Sobrevive a roubo/quebra da câmera	não	sim	sim (se o PC estiver em outro lugar)
Privacidade	fica em casa	fica nos servidores da TP-Link (criptografado)	fica em casa
Acesso de fora	pelo app (via nuvem, sem assinatura)	pelo app	só na rede local (ou túnel)
Quando usar	sempre (base)	se a câmera pode ser levada/danificada	laboratório/escola, integração com projetos

microSD

1. Use um cartão **classe 10 / U1 ou melhor**, de marca; formate **no app** (*Configurações → Cartão microSD → Formatar*). Versões antigas aceitam até 128 GB; as novas, 256–512 GB.
2. Escolha **Gravação contínua** (24 h) ou **por eventos**. Conta de capacidade: 1080p a ~1,5 Mbit/s ≈ 0,7 GB/h ≈ 16 GB/dia → um cartão de 128 GB guarda ~8 dias contínuos; por eventos, semanas.
3. **Loop**: quando enche, apaga o mais antigo automaticamente. Decida a **retenção** (LGPD): ex.: 7 dias.
4. **Playback**: tela ao vivo → *Reprodução*: linha do tempo com os eventos marcados; *Download* salva o trecho no celular (e dali para o Drive, se quiser).

Nuvem — Tapo Care

1. *Eu → Tapo Care* → teste grátis (30 dias) → depois plano mensal/anual (preços por região; confira no app). Grava **só eventos**, com notificações ricas e histórico de 7 ou 30 dias.
2. *Eu → Reprodução e download → Vídeo na nuvem* para rever, baixar e compartilhar.
3. Vantagem real: a prova sobrevive mesmo que levem a câmera. Desvantagem: dependência de assinatura e dados fora do país.

PC como NVR (local, sem assinatura)

A partir da Aula 4 você terá o endereço RTSP da câmera. O script abaixo grava o fluxo em arquivos de 10 min **ou** só quando há movimento (comparando quadros, como no desafio da aula de ESP32-CAM) e apaga arquivos com mais de N dias — um NVR de verdade em 70 linhas.

tapo_gravar.py

```
"""Gravador local (NVR caseiro) da camera Tapo: grava o RTSP em arquivos de N minutos e/ou so
quando ha movimento.

Uso:
    python tapo_gravar.py rtsp://u:s@192.168.1.50:554/stream1 --pasta gravacoes --minutos 10
# continuo, 1 arquivo a cada 10 min
    python tapo_gravar.py rtsp://u:s@192.168.1.50:554/stream2 --movimento
# so grava quando algo se move
    python tapo_gravar.py --webcam --movimento
# teste sem camera
    (--dias 7 apaga arquivos com mais de 7 dias: "gravacao em loop", como o microSD faz)

Por que isso existe: o microSD da camera e a nuvem Tapo Care sao as opcoes "de fabrica"; gravar
no PC por RTSP e a
terceira opcao, 100 % local, sem assinatura - e e o que um NVR/NAS faz. Formato AVI/MJPEG:
simples e sem codec extra.
"""
import sys, os, time, glob, cv2, numpy as np

args = sys.argv[1:]
def opt(nome, padrao=None):
    return args[args.index(nome) + 1] if nome in args and args.index(nome) + 1 < len(args) else
padrao
URL = args[0] if args and not args[0].startswith("--") else "--webcam"
PASTA = opt("--pasta", "gravacoes"); MINUTOS = float(opt("--minutos", "10")); DIAS =
float(opt("--dias", "0"))
MOVIMENTO = "--movimento" in args; LIMIAR_MOV = 1.5   # % de pixels que mudaram
os.makedirs(PASTA, exist_ok=True)
os.environ.setdefault("OPENCV_FFMPEG_CAPTURE_OPTIONS", "rtsp_transport;tcp")

def abrir(): return cv2.VideoCapture(0) if URL == "--webcam" else cv2.VideoCapture(URL,
cv2.CAP_FFMPEG)
def limpar_antigos():
    if DIAS <= 0: return
    for f in glob.glob(os.path.join(PASTA, "*.avi")):
        if time.time() - os.path.getmtime(f) > DIAS * 86400: os.remove(f); print("apagado
(antigo):", f)

cap = abrir()
if not cap.isOpened(): print("nao abriu", URL); sys.exit(1)
w, h, fps = int(cap.get(3)), int(cap.get(4)), cap.get(5) or 15
print(f"{w}x{h} @ {fps:.0f} fps -> {PASTA}/  modo: {'movimento' if MOVIMENTO else f'continuo
{MINUTOS:g} min/arquivo'}  (Ctrl+C para parar)")
gravador = None; inicio = 0; anterior = None; ultimo_mov = 0
try:
    while True:
        ok, q = cap.read()
        if not ok: time.sleep(1); cap.release(); cap = abrir(); continue
        agora = time.time()
        cv2.putText(q, time.strftime("%d/%m/%Y %H:%M:%S"), (10, h - 12),
cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)
        gravar = True
        if MOVIMENTO:
            cinza = cv2.GaussianBlur(cv2.cvtColor(cv2.resize(q, (320, 180)), cv2.COLOR_BGR2GRAY),
(21, 21), 0)
            if anterior is not None:
                diff = cv2.threshold(cv2.absdiff(anterior, cinza), 25, 255, cv2.THRESH_BINARY)[1]
                pct = 100.0 * np.count_nonzero(diff) / diff.size
                if pct > LIMIAR_MOV: ultimo_mov = agora
            anterior = cinza
            gravar = agora - ultimo_mov < 8          # continua 8 s apos o ultimo movimento
        # abre/fecha arquivos
        if gravar and (gravador is None or (not MOVIMENTO and agora - inicio > MINUTOS * 60)):
            if gravador: gravador.release()
            nome = os.path.join(PASTA, time.strftime("%Y-%m-%d_%H-%M-%S") + ("_mov" if MOVIMENTO
else "") + ".avi")
            gravador = cv2.VideoWriter(nome, cv2.VideoWriter_fourcc(*"MJPG"), fps, (w, h));
inicio = agora
            print("gravando", nome); limpar_antigos()
```

```
        if not gravar and gravador is not None: gravador.release(); gravador = None;
print("movimento acabou; arquivo fechado")
        if gravador: gravador.write(q)
except KeyboardInterrupt: pass
if gravador: gravador.release()
cap.release(); print("fim")
```

- `OPENCV_FFMPEG_CAPTURE_OPTIONS=rtsp_transport;tcp`: pede o vídeo por **TCP**; por UDP os quadros chegam "picotados" em Wi-Fi fraco.
- `--minutos 10`: um arquivo por 10 min (igual à câmera, que grava em blocos); `--dias 7`: retenção em loop; `--movimento`: diferença entre quadros (`absdiff`) acima de 1,5 % da imagem.
- `/stream2` (360p) é suficiente para "saber que houve movimento" e gasta 10× menos disco; use `/stream1` quando a qualidade importar.

Experimente

1. Grave 1 h contínua no cartão e veja no app quanto espaço usou; extrapole para 7 dias.
2. Baixe um clipe para o celular e abra no PC: resolução? codec? (H.264, ~1,5 Mbit/s)
3. Rode `tapo_gravar.py --movimento` por 20 min: quantos arquivos? Quantos são "falsos"?

Desafios (lógica)

1. **3.1** Política de retenção: escreva as regras (o que, onde, por quanto tempo, quem acessa, como apaga) para a câmera da sala — e justifique cada uma.
2. **3.2** Modifique `tapo_gravar.py` para gravar **1 min antes** do movimento (dica: guardar os últimos 900 quadros num `deque` e despejá-los no arquivo quando o movimento começar — é o *pre-record* dos NVRs).
3. **3.3** Faça o gravador só funcionar fora do horário de aula (`datetime.now().hour`).
4. **3.4** Calcule o custo total em 3 anos de: microSD 128 GB × Tapo Care × PC ligado 24 h (energia ~15 W). Qual você escolheria para a escola?

Aula 4 — A câmera na rede: RTSP, ONVIF e segurança

O que você vai aprender

- Criar a **conta da câmera** e descobrir/fixar o IP.
- Abrir o vídeo no **VLC** e no **Python/OpenCV** (RTSP) e integrar com NVR/NAS (ONVIF).
- Rede local × internet de novo: por que não expor a porta 554.
- Boas práticas: senha, 2FA, rede IoT, firmware.

RTSP e ONVIF em 1 minuto

RTSP (*Real Time Streaming Protocol*, porta **554**) é o protocolo padrão para pedir vídeo a uma câmera IP: `rtsp://usuario:senha@IP:554/stream1`. O vídeo vem em **H.264** — comprimido, 1080p em ~1,5 Mbit/s. A Tapo oferece dois fluxos: `/stream1` (1080p) e `/stream2` (360p, leve). **ONVIF** (porta **2020**) é um padrão da indústria para *descobrir e controlar* câmeras (pan/tilt, presets, configuração) — é o que NVRs e NAS (Synology, QNAP, Frigate, Home Assistant) usam para adicionar a câmera. Os dois exigem a **conta da câmera**, separada da conta TP-Link.

Passo a passo

1. No app: câmera → *Configurações → Avançado → Conta da câmera* → crie **usuário e senha** (anote; só letras e números evitam problemas na URL).
2. Descubra o IP: *Configurações → Informações do dispositivo* (ou no roteador). No roteador, **reserve o IP** pelo MAC da câmera (DHCP estático) — senão ele muda e os scripts param.
3. **VLC** (PC): *Mídia → Abrir fluxo de rede* → `rtsp://usuario:senha@192.168.1.50:554/stream1` → Reproduzir. Latência de 1–2 s é normal (buffer).
4. **Python**: `python tapo_ver.py rtsp://usuario:senha@192.168.1.50:554/stream2` — foto com `f`, gravação com `g`.
5. **ONVIF** (opcional): ative em *Avançado → ONVIF*; num NVR/NAS adicione a câmera por ONVIF em `http://IP:2020/onvif/device_service`.
6. Se falhar: feche o ao vivo no celular (a câmera limita conexões simultâneas), tente `/stream2`, confira usuário/senha e se o roteador isola clientes Wi-Fi (“AP isolation”).

tapo_ver.py

```
"""Ver o video da camera Tapo C200/TC70 no PC por RTSP (OpenCV), salvar foto e gravar trecho.

Antes: no app Tapo -> camera -> Configuracoes -> Avancado -> Conta da camera -> crie
usuario/senha.
Descubra o IP da camera no app (Informacoes do dispositivo) ou no roteador e, de preferencia,
fixe-o (DHCP reservado).

Uso:
    python tapo_ver.py rtsp://usuario:senha@192.168.1.50:554/stream1     # 1080p
    python tapo_ver.py rtsp://usuario:senha@192.168.1.50:554/stream2     # 360p (leve, ideal para
IA)
    python tapo_ver.py --webcam                                           # testa sem camera
Teclas: f = foto   g = gravar/parar (.avi)   q = sair
"""
import sys, time, os, cv2

os.environ.setdefault("OPENCV_FFMPEG_CAPTURE_OPTIONS", "rtsp_transport;tcp|stimeout;5000000")   #
TCP = sem "quadros picotados"
```

```
URL = sys.argv[1] if len(sys.argv) > 1 else "--webcam"
cap = cv2.VideoCapture(0 if URL == "--webcam" else URL, cv2.CAP_FFMPEG if URL != "--webcam" else
cv2.CAP_ANY)
if not cap.isOpened():
    print("nao abriu:", URL, "\n- PC na mesma rede da camera?  - conta da camera criada?  - IP
certo?  - outro programa usando o stream?"); sys.exit(1)
w, h, fps = int(cap.get(3)), int(cap.get(4)), cap.get(5) or 15
print(f"stream {w}x{h} @ {fps:.0f} fps  (f = foto, g = gravar, q = sair)")
gravador = None; t0 = time.time(); n = 0
while True:
    ok, q = cap.read()
    if not ok: print("quadro perdido; reconectando..."); time.sleep(1); cap.release(); cap =
cv2.VideoCapture(URL, cv2.CAP_FFMPEG); continue
    n += 1
    cv2.putText(q, time.strftime("%d/%m/%Y %H:%M:%S") + (f"   {n/(time.time()-t0):.1f} fps" if n
> 10 else "") + ("   REC" if gravador else ""),
                (10, h - 12), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255) if gravador else (255,
255, 255), 2)
    if gravador: gravador.write(q)
    cv2.imshow("Tapo (q sai)", q)
    k = cv2.waitKey(1) & 0xFF
    if k == ord("q"): break
    if k == ord("f"): nome = time.strftime("tapo_%Y%m%d_%H%M%S.jpg"); cv2.imwrite(nome, q);
print("foto", nome)
    if k == ord("g"):
        if gravador: gravador.release(); gravador = None; print("gravacao parada")
        else: nome = time.strftime("tapo_%Y%m%d_%H%M%S.avi"); gravador = cv2.VideoWriter(nome,
cv2.VideoWriter_fourcc(*"MJPG"), fps, (w, h)); print("gravando", nome)
if gravador: gravador.release()
cap.release(); cv2.destroyAllWindows()
```

- `cv2.VideoCapture(url, cv2.CAP_FFMPEG)`: o OpenCV usa o FFmpeg embutido para falar RTSP e decodificar H.264. Cada quadro vira a mesma matriz BGR das aulas anteriores — **todo o resto do curso de visão se aplica**.
- A diferença para o ESP32-CAM: aqui o vídeo já chega comprimido e estável; o PC gasta CPU para **decodificar** (1080p ≈ 10–20 % de um núcleo).

Segurança da câmera e da rede

- **Nunca** faça *port forwarding* da 554/2020 para a internet: câmeras expostas são encontradas em minutos (Shodan) e viram parte de botnets. Para acesso remoto use o app (P2P) ou um túnel (Tailscale) num PC da rede.
- Senha forte na conta TP-Link + **2FA**; senha diferente na conta da câmera; troque a senha padrão do roteador.
- **Rede separada para IoT** (SSID de convidados ou VLAN): se a câmera for invadida, não alcança os PCs.
- Mantenha o **firmware** atualizado (o app avisa). Desligue o ONVIF se não usar.
- O LED de status e o modo privacidade são garantias físicas para quem está na sala — não os desative para “esconder” a câmera.

Experimente

1. Abra `/stream1` e `/stream2` lado a lado no VLC: compare nitidez, atraso e uso de CPU (Gerenciador de Tarefas).
2. Com o VLC aberto, abra também o ao vivo no celular: algum dos dois trava? Quantas conexões a câmera aguenta?
3. No roteador, veja o tráfego da câmera (Mbit/s) parada e com movimento: H.264 gasta menos quando nada muda — por quê?

Desafios (lógica)

1. **4.1** Faça o `tapo_ver.py` tirar uma foto automaticamente a cada 60 s e montar um **timelapse** do dia (reuse o desafio da aula ESP32-CAM).
2. **4.2** Use ONVIF para **mover a câmera por código**: com a biblioteca `onvif-zeep` (`pip install onvif-zeep`), vá para o preset "Fundos" quando o Python detectar movimento na lateral (dica: `ptz.GotoPreset`). É a "patrulha inteligente".
3. **4.3** Monitore a saúde: um script que pinga a câmera a cada minuto e avisa (Telegram, como na aula de distância) se ela sumir da rede.
4. **4.4** Liste 5 diferenças entre a Tapo e o ESP32-CAM (custo, qualidade, abertura, consumo, integração). Quando cada um é a escolha certa?

Aula 5 — ESP32 atuador: luz, alarme e LED comandados pela rede

O que você vai aprender

- Um **relé** como interruptor controlado por software (e os cuidados com a rede elétrica).
- ESP32 em **modo estação** na mesma rede da câmera, com nome `luzes.local` (mDNS).
- Uma API HTTP simples (`/luz`, `/alerta`, `/led`, `/estado`) que qualquer programa pode chamar.
- Botão local como *fallback*: a luz tem que funcionar mesmo sem rede.

O relé

Um relé é um **eletroímã que fecha um contato**: 5 V na bobina (poucos mA, via transistor do módulo) fecham um contato que aguenta 10 A em 250 V — isolamento total entre o ESP32 e a carga. O módulo KY-019 tem **S** (sinal), **+** (5 V), **–** (GND) e, do outro lado, **COM** (comum), **NO** (normalmente aberto) e **NC** (normalmente fechado). A lâmpada vai em **COM–NO**: só acende quando o ESP32 manda. A maioria dos módulos liga com **sinal em LOW** (`RELE_ATIVO_BAIXO = true`). Na aula a "lâmpada" é um LED alimentado pelos 5 V através do contato — mesmo circuito, sem perigo.

Montagem

Componente	ESP32	Observação
Relé S / + / –	GPIO 26 / VIN / GND	VIN = 5 V do USB
Lâmpada (LED + 220 Ω)	COM ← 5 V; NO → resistor → LED → GND	em casa: lâmpada 12 V com fonte própria
Buzzer passivo	GPIO 27	
LED RGB R/G/B	GPIO 32 / 33 / 25	220 Ω cada; comum GND
Botão	GPIO 4 ↔ GND	INPUT_PULLUP

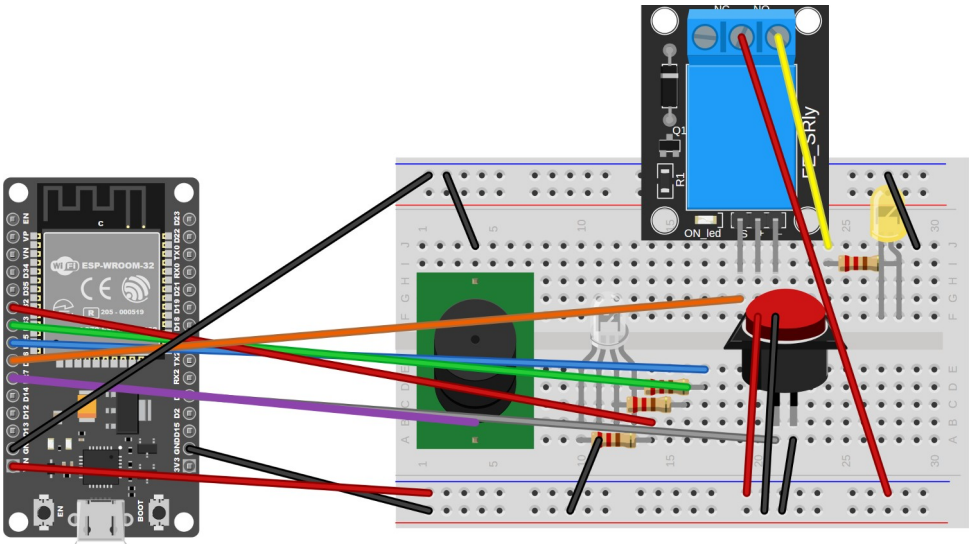


Figura 2 — Aula 5 — ESP32 com módulo relé (LED como lâmpada), buzzer, LED RGB e botão local.

Código

`01_esp32_luzes_alerta.ino`

```
/*
  01 - ESP32 "atuador" da camera Tapo: luz (rele), buzzer e LED RGB comandados pela rede
  Placa: ESP32 Dev Module (core 3.x)   Sem bibliotecas extras (WiFi, WebServer, ESPmDNS vem no
core)

  Ideia: a camera Tapo C200/TC70 NAO sabe acender luz. Quem decide e o PC (Python), que ve o
video da
  camera por RTSP, reconhece gestos/rostos e manda comandos HTTP para ESTE ESP32. Tudo na MESMA
rede Wi-Fi
  da camera (por isso o ESP32 entra na rede em MODO ESTACAO, nao cria a propria).

  Ligacoes:
    Modulo rele 1 canal (5 V): VCC -> VIN(5V)  GND -> GND  IN -> GPIO 26      (a luz 127/220 V
vai em COM/NO: SO COM O PROFESSOR)
         Para a aula, use uma lampada de 12 V ou uma fita de LED no lugar da rede eletrica!
    Buzzer passivo: S -> GPIO 27
    LED RGB catodo comum: R -> 220 -> GPIO 32   G -> 220 -> GPIO 33   B -> 220 -> GPIO 25   comum
-> GND
    Botao local (liga/desliga a luz na mao): GPIO 4 <-> GND

  Rotas (http://luzes.local ou http://<IP>):
    /                     pagina de estado com botoes
    /luz?on=1 | on=0      liga/desliga o rele      /luz?toggle=1
    /alerta?tipo=intruso  buzzer alarme + LED vermelho piscando por 10 s
    /alerta?tipo=bemvindo 2 bipes suaves + LED verde 5 s
    /alerta?tipo=off      silencia
    /led?cor=verde        LED RGB (verde, vermelho, azul, amarelo, roxo, ciano, branco, off)
    /estado               JSON {"luz":true,"alerta":"intruso","ultimo":"..."}
*/
#include <WiFi.h>
#include <WebServer.h>
#include <ESPmDNS.h>

const char* WIFI_SSID = "NOME-DA-REDE";       // a MESMA rede da camera Tapo (2,4 GHz)
const char* WIFI_SENHA = "SENHA-DA-REDE";
const char* NOME_MDNS = "luzes";              // http://luzes.local
const int PINO_RELE = 26, PINO_BUZZER = 27, PINO_R = 32, PINO_G = 33, PINO_B = 25, PINO_BOTAO =
4;
const bool RELE_ATIVO_BAIXO = true;           // a maioria dos modulos de rele liga com IN em LOW

WebServer servidor(80);
bool luzLigada = false;
String alertaAtual = "off", ultimoComando = "nenhum";
unsigned long fimAlerta = 0, ultimoBipe = 0;
bool bipando = false, botaoAntes = HIGH;

void cor(int r, int g, int b) { ledcWrite(PINO_R, r); ledcWrite(PINO_G, g); ledcWrite(PINO_B, b);
}
bool corPorNome(const String& n) {
  if (n == "verde") cor(0, 255, 0); else if (n == "vermelho") cor(255, 0, 0); else if (n ==
"azul") cor(0, 0, 255);
  else if (n == "amarelo") cor(255, 150, 0); else if (n == "roxo") cor(160, 0, 255); else if (n
== "ciano") cor(0, 255, 255);
  else if (n == "branco") cor(255, 255, 255); else if (n == "off") cor(0, 0, 0); else return
false;
  return true;
}
void setLuz(bool on) { luzLigada = on; digitalWrite(PINO_RELE, (on != RELE_ATIVO_BAIXO) ? HIGH :
LOW); Serial.printf("luz -> %s\n", on ? "ON" : "OFF"); }
void setAlerta(const String& tipo) {
  alertaAtual = tipo; noTone(PINO_BUZZER); bipando = false;
  if (tipo == "intruso")  { fimAlerta = millis() + 10000; cor(255, 0, 0); }
  else if (tipo == "bemvindo") { fimAlerta = millis() + 5000; cor(0, 255, 0); tone(PINO_BUZZER,
1200, 120); delay(160); tone(PINO_BUZZER, 1800, 160); }
  else { fimAlerta = 0; cor(0, 0, 0); alertaAtual = "off"; }
  Serial.println("alerta -> " + alertaAtual);
}

const char PAGINA[] PROGMEM = R"HTML(<!DOCTYPE html><html lang="pt-br"><head><meta
charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1">
<title>Luzes e alertas</title><style>body{font-family:sans-serif;background:#111;color:#eee;text-
align:center;padding:14px}
.card{background:#222;border-radius:12px;padding:14px;margin:10px auto;max-width:420px}
button{font-size:1em;padding:10px 16px;margin:5px;border:0;border-radius:8px;color:#fff}
```

```
.on{background:#2a2}.off{background:#555}.al{background:#c22}.bv{background:#36c}#luz{font-
size:1.6em}</style></head><body>
<h2>ESP32 — luzes e alertas (Tapo)</h2><div class="card"><div id="luz">luz: --</div>
<button class="on" onclick="c('/luz?on=1')">Ligar luz</button><button class="off"
onclick="c('/luz?on=0')">Desligar</button></div>
<div class="card"><button class="al" onclick="c('/alerta?tipo=intruso')">Alarme
intruso</button><button class="bv" onclick="c('/alerta?tipo=bemvindo')">Boas-vindas</button>
<button class="off" onclick="c('/alerta?tipo=off')">Silenciar</button><p id="st"></p></div>
<script>async function c(u){await fetch(u);e()}async function e(){const d=await (await
fetch('/estado')).json();
document.getElementById('luz').textContent='luz: '+
(d.luz?'LIGADA':'desligada');document.getElementById('st').textContent='alerta: '+d.alerta+' |
ultimo comando: '+d.ultimo}
e();setInterval(e,1500)</script></body></html>)HTML";

void rotaPagina() { servidor.send_P(200, "text/html; charset=utf-8", PAGINA); }
void rotaLuz() {
  if (servidor.hasArg("toggle")) setLuz(!luzLigada); else if (servidor.hasArg("on"))
setLuz(servidor.arg("on") == "1");
  ultimoComando = "luz " + String(luzLigada ? "on" : "off");
  servidor.sendHeader("Access-Control-Allow-Origin", "*"); servidor.send(200, "text/plain",
luzLigada ? "on" : "off");
}
void rotaAlerta() { setAlerta(servidor.arg("tipo")); ultimoComando = "alerta " + alertaAtual;
servidor.sendHeader("Access-Control-Allow-Origin", "*"); servidor.send(200, "text/plain",
alertaAtual); }
void rotaLed() { corPorNome(servidor.arg("cor")); ultimoComando = "led " + servidor.arg("cor");
servidor.sendHeader("Access-Control-Allow-Origin", "*"); servidor.send(200, "text/plain",
"ok"); }
void rotaEstado() {
  String j = "{\"luz\":" + String(luzLigada ? "true" : "false") + ",\"alerta\":\"" + alertaAtual
+ "\",\"ultimo\":\"" + ultimoComando + "\",\"ip\":\"" + WiFi.localIP().toString() + "\"}";
  servidor.sendHeader("Access-Control-Allow-Origin", "*"); servidor.send(200, "application/json",
j);
}

void setup() {
  Serial.begin(115200);
  pinMode(PINO_RELE, OUTPUT); setLuz(false);
  pinMode(PINO_BUZZER, OUTPUT); pinMode(PINO_BOTAO, INPUT_PULLUP);
  ledcAttach(PINO_R, 5000, 8); ledcAttach(PINO_G, 5000, 8); ledcAttach(PINO_B, 5000, 8);
  WiFi.mode(WIFI_STA); WiFi.begin(WIFI_SSID, WIFI_SENHA);
  Serial.print("Conectando"); cor(0, 0, 60);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.print("\nIP: "); Serial.println(WiFi.localIP());
  if (MDNS.begin(NOME_MDNS)) Serial.printf("http://%s.local\n", NOME_MDNS);
  servidor.on("/", rotaPagina); servidor.on("/luz", rotaLuz); servidor.on("/alerta", rotaAlerta);
servidor.on("/led", rotaLed); servidor.on("/estado", rotaEstado);
  servidor.begin();
  cor(0, 0, 0); tone(PINO_BUZZER, 1500, 80);
}

void loop() {
  servidor.handleClient();
  // botao local: liga/desliga a luz mesmo sem rede
  bool b = digitalRead(PINO_BOTAO);
  if (botaoAntes == HIGH && b == LOW) { setLuz(!luzLigada); delay(50); }
  botaoAntes = b;
  // alerta "intruso": buzzer sirene + LED piscando ate o fim
  if (alertaAtual == "intruso") {
    if (millis() > fimAlerta) setAlerta("off");
    else if (millis() - ultimoBipe > 250) { ultimoBipe = millis(); bipando = !bipando; if
(bipando) { tone(PINO_BUZZER, 2200); cor(255, 0, 0); } else { tone(PINO_BUZZER, 1400); cor(0, 0,
0); } }
  } else if (alertaAtual == "bemvindo" && millis() > fimAlerta) setAlerta("off");
}
```

Entenda o código

- `WiFi.mode(WIFI_STA)` + `WiFi.begin`: o ESP32 **entra** na rede (como a câmera), em vez de criar a própria. `MDNS.begin("luzes")` dá o nome `luzes.local` — o Python não precisa saber o IP.

- Rotas: `/luz?on=1`, `/alerta?tipo=intruso|bemvindo|off`, `/led?cor=`, `/estado` (JSON). `Access-Control-Allow-Origin: *` permite que páginas de outros endereços chamem.
- `setAlerta("intruso")` arma um **temporizador** (`fimAlerta`); o `loop()` faz a sirene (dois tons alternados) sem `delay` e desarma sozinho — nunca deixe um alarme depender de alguém mandar "parar".
- O botão físico inverte a luz mesmo se o Wi-Fi cair: **o controle local sempre existe**.

Experimente

1. Abra `http://luzes.local` no celular (mesma rede) e use os botões. Depois `http://luzes.local/estado` cru.
2. Mande `curl http://luzes.local/luz?toggle=1` do PC. Quanto tempo leva?
3. Inverta `RELE_ATIVO_BAIXO`: o que acontece ao ligar o ESP32? (o relé pode "clicar" no boot — bom para discutir estado inicial seguro).

Desafios (lógica + alertas)

1. **5.1 Temporizador de luz** — `/luz?on=1&min=5`: acende e apaga sozinha após 5 min (e o botão cancela).
2. **5.2 Dois relés** — luz e ventilador; rota `/cena?nome=aula|vazia|noite` aplica combinações (tabela de cenas).
3. **5.3 Proteção** — se a rota `/alerta?tipo=intruso` for chamada 5 vezes em 1 min, ignore (anti-spam) e pisque amarelo.
4. **5.4 Estado persistente** — guarde o estado da luz em `Preferences` e restaure após queda de energia.
5. **5.5 Página com histórico** — mostre na página os últimos 10 comandos com hora (NTP).

Aula 6 — Projeto: gestos e reconhecimento facial acionam o ESP32

O que você vai aprender

- Juntar tudo: Tapo (olhos) → Python (cérebro) → ESP32 (mãos).
- **Reconhecimento facial** de verdade: detectar (YuNet) → alinhar → **vetor de 128 números** (SFace) → comparar (cosseno).
- Cadastro consentido de pessoas; limiar de decisão; falsos positivos/negativos.
- Regras de automação: gestos para comandos, rosto conhecido = boas-vindas, desconhecido = alarme — com cooldown.

Como funciona o reconhecimento facial

Detectar (onde há um rosto) e **reconhecer** (de quem é) são problemas diferentes. O **YuNet** é uma rede neural minúscula que encontra rostos e 5 pontos (olhos, nariz, cantos da boca). O **SFace** recebe o rosto alinhado por esses pontos e produz um **embedding**: 128 números que resumem a geometria do rosto — rostos da mesma pessoa dão vetores *parecidos* (similaridade de cosseno > 0,363, o limiar recomendado pelo OpenCV), de pessoas diferentes dão vetores distantes. Cadastrar = guardar os vetores com o nome; reconhecer = comparar com todos e pegar o mais parecido. **Nenhuma foto fica guardada**, só números — e isso importa para a privacidade.

rostos.py (módulo)

```
"""Reconhecimento facial com OpenCV (YuNet detecta, SFace reconhece) - modulo usado por
cadastrar_rosto.py e tapo_gestos_faces.py.

Como funciona:
  1. YuNet (rede neural leve) encontra rostos na imagem e 5 pontos (olhos, nariz, boca).
  2. SFace alinha o rosto por esses pontos e gera um "vetor de caracteristicas" de 128 numeros
(embedding).
  3. Reconhecer = comparar o vetor do rosto novo com os vetores cadastrados (similaridade de
cosseno).
     > 0,363 = mesma pessoa (limiar recomendado pelo OpenCV). Nenhuma imagem e guardada: so os
vetores.
Modelos (python/modelos/): face_detection_yunet_2023mar.onnx  e
face_recognition_sface_2021dec.onnx
"""
import os, cv2, numpy as np

PASTA = os.path.dirname(os.path.abspath(__file__))
MODELOS = os.path.join(PASTA, "modelos")
BANCO = os.path.join(PASTA, "rostos_cadastrados.npz")
LIMIAR_COS = 0.363

class Reconhecedor:
    def __init__(self, tamanho=(320, 320)):
        self.det = cv2.FaceDetectorYN.create(os.path.join(MODELOS,
"face_detection_yunet_2023mar.onnx"), "", tamanho, 0.8, 0.3, 5000)
        self.rec = cv2.FaceRecognizerSF.create(os.path.join(MODELOS,
"face_recognition_sface_2021dec.onnx"), "")
        self.nomes, self.vetores = [], np.zeros((0, 128), np.float32)
        self.carregar()
    def carregar(self):
        if os.path.exists(BANCO):
            d = np.load(BANCO, allow_pickle=True); self.nomes = list(d["nomes"]); self.vetores =
d["vetores"].astype(np.float32)
    def salvar(self): np.savez(BANCO, nomes=np.array(self.nomes), vetores=self.vetores)
    def detectar(self, img):
        """devolve lista de rostos: cada um e um vetor [x, y, w, h, 5 pontos..., confianca]"""
        self.det.setInputSize((img.shape[1], img.shape[0]))
        _, rostos = self.det.detect(img)
        return [] if rostos is None else rostos
    def vetor(self, img, rosto):
        alinhado = self.rec.alignCrop(img, rosto)
```

```
        return self.rec.feature(alinhado)[0]
    def cadastrar(self, nome, img, rosto):
        self.nomes.append(nome); self.vetores = np.vstack([self.vetores, self.vetor(img, rosto)
[None, :]]); self.salvar()
    def quem(self, img, rosto):
        """devolve (nome ou 'desconhecido', similaridade)"""
        if len(self.nomes) == 0: return "desconhecido", 0.0
        v = self.vetor(img, rosto)
        v = v / (np.linalg.norm(v) + 1e-9); base = self.vetores / (np.linalg.norm(self.vetores,
axis=1, keepdims=True) + 1e-9)
        sims = base @ v; i = int(np.argmax(sims))
        return (self.nomes[i], float(sims[i])) if sims[i] > LIMIAR_COS else ("desconhecido",
float(sims[i]))
    def desenhar(self, img, rosto, texto, cor):
        x, y, w, h = [int(v) for v in rosto[:4]]
        cv2.rectangle(img, (x, y), (x + w, y + h), cor, 2)
        cv2.putText(img, texto, (x, max(20, y - 8)), cv2.FONT_HERSHEY_SIMPLEX, 0.7, cor, 2)
```

Cadastro

cadastrar_rosto.py

```
"""Cadastra o rosto de uma pessoa (varias fotos) no banco rostos_cadastrados.npz.

Uso:
    python cadastrar_rosto.py "Maria" rtsp://u:s@192.168.1.50:554/stream2
    python cadastrar_rosto.py "Maria" --webcam
    python cadastrar_rosto.py --listar          python cadastrar_rosto.py --apagar "Maria"
Na janela: fique de frente, boa luz; aperte ESPACO para cadastrar a pose atual (faca 5-8: frente,
leve giro, sorrindo, oculos...). q sai.
Privacidade: o arquivo guarda so o NOME e 128 numeros por foto - nao guarda a imagem. Cadastre
apenas quem consentiu.
"""
import sys, os, cv2
from rostos import Reconhecedor

args = sys.argv[1:]
r = Reconhecedor()
if "--listar" in args:
    from collections import Counter
    for n, c in Counter(r.nomes).items(): print(f"  {n}: {c} amostra(s)")
    sys.exit()
if "--apagar" in args:
    nome = args[args.index("--apagar") + 1]; keep = [i for i, n in enumerate(r.nomes) if n !=
nome]
    r.nomes = [r.nomes[i] for i in keep]; r.vetores = r.vetores[keep]; r.salvar();
print("apagado", nome); sys.exit()
nome, fonte = args[0], (args[1] if len(args) > 1 else "--webcam")
os.environ.setdefault("OPENCV_FFMPEG_CAPTURE_OPTIONS", "rtsp_transport;tcp")
cap = cv2.VideoCapture(0) if fonte == "--webcam" else cv2.VideoCapture(fonte, cv2.CAP_FFMPEG)
n = 0
while True:
    ok, q = cap.read()
    if not ok: continue
    rostos = r.detectar(q)
    for f in rostos: r.desenhar(q, f, f"{nome}? ESPACO cadastra", (0, 200, 255))
    cv2.putText(q, f"amostras de {nome}: {n}   (ESPACO = cadastrar, q = sair)", (10, 28),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)
    cv2.imshow("cadastro", q)
    k = cv2.waitKey(1) & 0xFF
    if k == ord("q"): break
    if k == 32 and len(rostos) == 1: r.cadastrar(nome, q, rostos[0]); n += 1; print("cadastrada
amostra", n)
    elif k == 32: print("precisa de exatamente 1 rosto na imagem")
cap.release(); cv2.destroyAllWindows(); print(f"{nome}: {n} amostras salvas em
{os.path.basename(r.__module__)}")
```

`python cadastrar_rosto.py "Maria" rtsp://u:s@IP:554/stream2` (ou `--webcam`): fique de frente para a câmera e aperte **ESPAÇO** 5–8 vezes mudando levemente a pose. `--listar` mostra quem está no banco; `--apagar "Maria"` remove (direito de exclusão!).

O projeto

tapo_gestos_faces.py

```
"""Projeto final: a camera Tapo ve -> o PC reconhece gestos (MediaPipe) e rostos (OpenCV SFace)
-> o ESP32 age.

Uso:
    python tapo_gestos_faces.py rtsp://u:s@192.168.1.50:554/stream2 --esp luzes.local
    python tapo_gestos_faces.py --webcam --esp 192.168.1.80
    python tapo_gestos_faces.py --webcam --sem-esp            # so reconhece
Regras (mude em REGRAS):
    gesto Open_Palm  -> liga a luz        Closed_Fist -> desliga        Thumb_Up -> LED verde
Victory -> LED azul
    rosto CONHECIDO  -> "bemvindo" (2 bipes + verde) uma vez a cada 60 s por pessoa
    rosto DESCONHECIDO por 3 s seguidos -> alerta "intruso" (sirene + vermelho) e foto salva em
alertas/
    ninguem -> nada.  Tecla q sai.
Modelos: modelos/gesture_recognizer.task (copie da aula ESP32-CAM) + modelos/*.onnx (rostos).
"""
import sys, os, time, threading, cv2, numpy as np, requests
import mediapipe as mp
from mediapipe.tasks.python import vision, BaseOptions
from rostos import Reconhecedor


args = sys.argv[1:]
FONTE = args[0] if args and not args[0].startswith("--") else "--webcam"
ESP = args[args.index("--esp") + 1] if "--esp" in args else "luzes.local"
ENVIAR = "--sem-esp" not in args
PASTA = os.path.dirname(os.path.abspath(__file__))
MODELO_GESTOS = os.path.join(PASTA, "modelos", "gesture_recognizer.task")
REGRAS = {"Open_Palm": "/luz?on=1", "Closed_Fist": "/luz?on=0", "Thumb_Up": "/led?cor=verde",
"Victory": "/led?cor=azul", "Thumb_Down": "/alerta?tipo=off"}
TEMPO_INTRUSO = 3.0; COOLDOWN_BEMVINDO = 60; COOLDOWN_INTRUSO = 30
os.makedirs(os.path.join(PASTA, "alertas"), exist_ok=True)


def enviar(rota):
    if not ENVIAR: return
    def _go():
        try: requests.get(f"http://{ESP}{rota}", timeout=2); print("ESP32 <-", rota)
        except requests.RequestException: print("(ESP32 nao respondeu em", ESP, ")")
    threading.Thread(target=_go, daemon=True).start()


gestos =
vision.GestureRecognizer.create_from_options(vision.GestureRecognizerOptions(base_options=BaseOpt
ions(model_asset_path=MODELO_GESTOS), num_hands=1))
rec = Reconhecedor()
os.environ.setdefault("OPENCV_FFMPEG_CAPTURE_OPTIONS", "rtsp_transport;tcp")
cap = cv2.VideoCapture(0) if FONTE == "--webcam" else cv2.VideoCapture(FONTE, cv2.CAP_FFMPEG)
if not cap.isOpened(): print("nao abriu", FONTE); sys.exit(1)
print(f"fonte {FONTE} | ESP32 {ESP} | pessoas cadastradas: {sorted(set(rec.nomes))}")


hist_gesto, ultimo_gesto_enviado = [], None
desconhecido_desde = None; ultimo_bemvindo = {}; ultimo_intruso = 0
while True:
    ok, q = cap.read()
    if not ok: time.sleep(0.5); continue
    agora = time.time()
    # ---- gestos ----
    res = gestos.recognize(mp.Image(image_format=mp.ImageFormat.SRGB, data=cv2.cvtColor(q,
cv2.COLOR_BGR2RGB)))
    gesto = res.gestures[0][0].category_name if res.gestures else "None"
    hist_gesto = (hist_gesto + [gesto])[-5:]
    if hist_gesto.count(gesto) >= 4 and gesto in REGRAS and gesto != ultimo_gesto_enviado:
        enviar(REGRAS[gesto]); ultimo_gesto_enviado = gesto
    if gesto == "None": ultimo_gesto_enviado = None
    # ---- rostos ----
    rostos = rec.detectar(q); ha_desconhecido = False
    for f in rostos:
        nome, sim = rec.quem(q, f)
```

```
        if nome == "desconhecido":
            ha_desconhecido = True; rec.desenhar(q, f, f"desconhecido ({sim:.2f})", (0, 0, 255))
        else:
            rec.desenhar(q, f, f"{nome} ({sim:.2f})", (0, 255, 0))
            if agora - ultimo_bemvindo.get(nome, 0) > COOLDOWN_BEMVINDO: ultimo_bemvindo[nome] =
agora; enviar("/alerta?tipo=bemvindo"); enviar("/luz?on=1")
    if ha_desconhecido:
        desconhecido_desde = desconhecido_desde or agora
        if agora - desconhecido_desde > TEMPO_INTRUSO and agora - ultimo_intruso >
COOLDOWN_INTRUSO:
            ultimo_intruso = agora; enviar("/alerta?tipo=intruso")
            foto = os.path.join(PASTA, "alertas", time.strftime("intruso_%Y%m%d_%H%M%S.jpg"));
cv2.imwrite(foto, q); print("foto salva", foto)
    else: desconhecido_desde = None
    cv2.putText(q, f"gesto: {gesto}   rostos: {len(rostos)}", (10, 28), cv2.FONT_HERSHEY_SIMPLEX,
0.8, (255, 255, 0), 2)
    cv2.imshow("Tapo + ESP32 (q sai)", q)
    if cv2.waitKey(1) & 0xFF == ord("q"): break
cap.release(); cv2.destroyAllWindows()
```

Entenda o sistema

- A fonte é o `/stream2` (360p): suficiente para gestos e rostos a até ~3 m e leve para o PC. Para rostos mais distantes, `/stream1`.
- **Gestos** (MediaPipe, da aula ESP32-CAM): votação 4/5 quadros, e a regra só dispara **na mudança** de gesto (`ultimo_gesto_enviado`) — mão aberta parada não fica religando a luz.
- **Rostos**: conhecido → `bemvindo` + luz, com **cooldown de 60 s por pessoa**; desconhecido por **3 s seguidos** → `intruso` + foto em `alertas/` + cooldown de 30 s. Os tempos evitam alarmes por um quadro ruim.
- Os envios ao ESP32 rodam em threads: o vídeo não trava esperando a rede.
- Tudo é **configurável em `REGRAS`**: é uma tabela gesto → rota, como as tabelas de estados das aulas anteriores.

Roteiro de teste

1. Grave o sketch 01 no ESP32 e confirme `http://luzes.local/estado`.
2. Cadastre 2–3 pessoas do grupo (com consentimento). Teste `python tapo_gestos_faces.py --webcam --sem-esp` primeiro.
3. Troque para a Tapo (`rtsp://…/stream2`). Aponte-a para a porta (preset “Porta”).
4. Entre na sala: boas-vindas + luz. Peça a alguém não cadastrado: após 3 s, sirene e foto.
5. Mão aberta: luz acende; punho: apaga; V: LED azul. Meça a latência total (gesto → luz).

Desafios (lógica + alertas)

1. **6.1 Modo armado** — gesto *Thumb_Down* arma o sistema (LED vermelho fixo); desarmado, desconhecidos **não** disparam o alarme (visitas durante a aula).
2. **6.2 Limiar** — varie `LIMIAR_COS` (0,30 / 0,363 / 0,45) e monte uma tabela de **falsos positivos × falsos negativos** com 2 pessoas cadastradas e 2 não. Qual limiar você escolheria para uma porta de laboratório? E para um cofre?
3. **6.3 Dois ESP32** — o da luz (`luzes.local`) e o do braço de servos (`braco.local`): rosto conhecido = braço acena.

4. **6.4 Registro** — cada evento (gesto, nome, intruso) numa linha de CSV com hora; gráfico de presença por hora no fim do dia.
5. **6.5 Alerta na nuvem** — ao detectar intruso, envie a foto por Telegram (reuse a aula ESP32-CAM, agora a partir do Python, com `requests.post(sendPhoto)`).
6. **6.6 Patrulha inteligente** — junte com ONVIF (desafio 4.2): ao ouvir movimento no preset “Lateral D”, a câmera vai até lá, e o Python só roda rostos quando a câmera parar de mover.

Para ir além

- **Home Assistant + Frigate**: NVR com detecção de objetos (pessoa/carro/animal) por ONVIF/RTSP, integrando a Tapo, o ESP32 e tudo o mais numa única central.
- **Múltiplas câmeras**: duas Tapo (lateral e fundos) no mesmo script; mapa de ocupação da sala.
- **Reconhecimento no ESP32-S3**: detecção de rostos embarcada, sem PC.
- **Ética em IA**: viés em reconhecimento facial (desempenho por tom de pele/idade), consentimento, uso em escolas — discussão obrigatória antes de qualquer uso real.

Glossário

Termo	Significado
Pan / tilt / preset / patrulha	giro horizontal / vertical / posição salva / passeio automático pelos presets
IR 850 nm	infravermelho próximo usado na visão noturna; invisível, reflete em vidro
H.264	codec de vídeo que comprime guardando só o que muda entre quadros
RTSP / ONVIF	protocolo de fluxo de vídeo (554) / padrão de descoberta e controle de câmeras (2020)
Conta da câmera	usuário/senha locais para RTSP/ONVIF, diferentes da ID TP-Link
P2P / NAT	como o app chega à câmera de fora sem abrir portas / o roteador que esconde a rede
Tapo Care	nuvem paga de clipes de eventos (7/30 dias)
NVR / NAS	gravador de vídeo em rede / armazenamento em rede
Gravação em loop / retenção	sobrescrever o mais antigo / por quanto tempo guardar
Relé / NO / NC / COM	interruptor eletromecânico / normalmente aberto / fechado / comum
mDNS	nomes `.local` na rede, sem servidor DNS
YuNet / SFace / embedding	detector de rostos / modelo que gera o vetor de 128 números / o vetor
Similaridade de cosseno	medida de parecença entre vetores (1 = iguais)
Cooldown	tempo mínimo entre dois alertas iguais
LGPD	lei brasileira de proteção de dados pessoais (imagem é dado pessoal)

Anexo — Checklist de instalação da câmera

- [ ] Ponto escolhido cobre lateral direita + fundos; sem contraluz; Wi-Fi 2,4 GHz com sinal
- [ ] Firmware atualizado; fuso horário; imagem invertida se necessário
- [ ] Presets Fundos / Lateral D / Porta e patrulha configurados
- [ ] Zonas e horário de detecção; notificações testadas
- [ ] microSD formatado; modo de gravação e retenção definidos
- [ ] Conta da câmera criada; IP reservado no roteador; VLC abre `/stream1`
- [ ] 2FA na ID TP-Link; porta 554 **não** exposta; cartaz de ambiente monitorado
- [ ] Acordo de privacidade da turma assinado; pessoas cadastradas consentiram